4EYES Labs Audit

Lithos: Mining (Part 1)

Auditor: Luca D'Angelo

Date: 2026-08-12

Table of Contents

Disclaimer

This audit is provided as-is at the time of its completion and might not reflect the most up-to-date state of any smart-contracts deployed, or any design document, or any research paper referenced herein.

Audit Summary

Protocol	Lithos: Mining (Part 1)
Repository	https://github.com/Lithos-Protocol/Lithos-Client
Commit	d6c400c70b6e06ebdfd2e498d5d3924c4acf38f3

Contract	File Name	Severity	Status
LIT Emissions	LIT_Emissions.ergo	WARNING	RESOLVED
Emission Config	Emission_Config.ergo	WARNING	RESOLVED
Collateral Enforcer	Collateral_Enforcer.ergo	WARNING	RESOLVED
Emission Gate	Emission_Gate.ergo	GOOD	RESOLVED
Emission Guard	Emission_Guard.ergo	GOOD	RESOLVED
Collateral Mainnet	Collateral_Mainnet.ergo	GOOD	RESOLVED

Contracts

LIT Emissions

Description

- The contract that allows lenders to provide their collateral for the pool and releases the LIT emissions for each block.

Box Contents

Registers

- R4 contains currentBlock as an Int representing the current position in the emission schedule.
- R5 contains lenderSet as a Coll[Coll[Byte]] representing a list of hashed lender proposition bytes that hold distinct live collateral boxes. The size of the list never exceeds CONST_MAX_ACTIVE, i.e. different PKs, but not necessarily different people.
- R6 contains queueHead as a Long representing the position of the queue box to be processed next.
- R7 contains queueTail as a Long representing the position that the next queue box created will be given.

Tokens

- Index 0: The emissionNFT representing the emission box.
- Index 1: The queueTokens placed in a queue box and burned when the box leaves the queue.
- Index 2: The collateralTokens placed in every collateral box, which also gets transferred to its proof-of-spend box.
- Index 3: The tokenLIT (LIT) tokens, placed at the end so that when emissions goes to zero, the token array does not require any shifting.

Context Variables

- Index 0: CTX_OP is a Byte representing the operation intended to be taken.
    - 0 is Join representing locking a lender's principal and permit into a queue box at the tail of the queue.
    - 1 is Activate representing turning the head queue box into a collateral box, where the block's LIT emission is added.
    - 2 is Clear representing returning the head queue box to its corresponding lender, which can only happen if the lender has a live collateral box so it cannot be added into the lenderSet again.
- Index 1: CTX_SLOT_IDX is an Int representing the index in lenderSet the lender of the head queue box occupies, only under the Activate operation, otherwise it defaults to -1.

Compile-Time Constants

- CONST_COLLATERAL_HASH: a Coll[Byte] representing the hashed proposition bytes of the collateral contract.
- CONST_GATE_HASH: a Coll[Byte] representing the hashed proposition bytes of the emission gate contract guarding the queue and proof-of-spend boxes.
    - This means that both of these boxes have the same Ergotree. Depending on the spending path, the emission contract treats the box as either a queue box or proof-of-spend box, expecting corresponding register data depending on the box type.
- CONST_LIT_ID: a Coll[Byte] representing the token id of the LIT token.
- CONST_F1: a Coll[Byte] representing the hashed proposition bytes of the Team address.
- CONST_F2: a Coll[Byte] representing the hashed proposition bytes of the Private Investor address.
- CONST_F3: a Coll[Byte] representing the hashed proposition bytes of the Auditors.

Contract Analysis

- L65-L96 defines the LIT token emission information as hard-coded constants.
    - The parameters match with the publicly announced tokenomics here: https://x.com/LithosProtocol/status/2075412442713129391
- L115-L118 defines the emissionConfig data-input box at index 0:
    - R7 is expected to contain the hash of the rollup-holding contract, assigned to the rollupHoldingHash variable. Nothing is enforced yet.
- L121-L143 defines the expected values of the output emission box, at index 0, to corresponding variables.
- L169 defines the current epoch as the quotient of currentBlock and CONST_EPOCH_LENGTH, assigning it to the currentEpoch variable.
- L170-L175 checks the current private rate and assigns it to the currentPrivRate variable.
    - The private rate is zero once the current epoch exceeds the private epoch length.
- L176-L188 checks the current emission rate and assigns it to the currentEmissionRate variable.
    - If the current block exceeds the set expected final block then the emission is set to 0.
    - If the current epoch is less than the phase 2 start then the emission is set to decrease by CONST_PUB_DECAY every CONST_P1_DECAY_LENGTH epochs, i.e. every integer epoch the emission is decreased by (CONST_PUB_DECAY / CONST_P1_DECAY_LENGTH)
    - If the current epoch is less than the phase 3 but:
        - The current epoch equals the phase 2 start, then the first phase 2 rate is set.
        - Otherwise, the second phase 2 rate is set.
    - Otherwise, the phase 3 rate is set.

- L189 defines totalEmissionRate as the sum of the current emission rate and the current private rate.
- L195-L200 defines emittedTotal checking that all remaining LIT tokens are emitted if the amount is below the calculated total emission rate. Otherwise, use the totalEmissionRate value.
- L203-L208 defines emittedPriv checking that the private rate is 0 if the emittedTotal is not enough to account for the private rate.
- L198 defines emittedPub, the public rate, as the difference of emittedTotal and emittedPriv.
- L213-L218 defines the foundersMap containing a list of tuples, each representing a pair of the hashed proposition bytes and the corresponding rate amount.
- L224-L233 defines a helper method called totalInInputs for checking the total amount of tokens of a given tokenId in the input boxes of the transaction.
- L237 defines onlyOne checking that the id of the first input box at index 0 is the id of the current box.
- L238 defines emissionBoxPreserved checking that:
    - onlyOne is true.
    - The output emission box preserves the following from the input emission box:
        - The proposition bytes.
        - The value.
        - The emission NFT.
        - The queue tokens, but it could have different amounts.
        - The collateral tokens, but it could have different amounts.
- L250 checks for the Join transaction where:
    - L251 defines queueOut as the second output box at index 1.
    - L252 defines queueLIT as the LIT token at index 1 in the output queueOut box or has a fallback to a default value.
        - This represents the LIT tokens provided by the lender as their permit for providing their collateral ERG in the output queue box.
    - L254 defines validQueueBox checking that:
        - The hash of the queueOut box propositionBytes equals the CONST_GATE_HASH compile-time constant.
        - The queueOut box has exactly 1 queue token at index 0.
        - The queueOut box has at most 2 different tokens.
        - The queueLIT token has as id the CONST_LIT_ID compile-time constant or the amount is 0 coming from the default fallback.
        - R5 of the queueOut box has a SigmaProp value defined, which represents the lenderPk.
        - R4 of the queueOut box has a Long value defined, which represents the fee value.
        - R6 of the queueOut box has a Byte value defined, which represents the extension flag.
    - L269 defines queueGrown checking that:

        - The total amount of queue tokens in the inputs equals the amount of queue tokens in the emission box.
        - The total amount of collateral tokens in the inputs equals the amount of collateral tokens in the emission box.
        - The amount of queue tokens in the output emission box is equal to the amount in the input emission box minus 1.
        - The amount of collateral tokens in the output emission box is equal to the amount in the input emission box.
        - The block increment, lender set, and LIT tokens in the output emission box are the same as in the input emission box.
        - The output emission box has the same amount of unique tokens as the input emission box.
    - L286 checks that emissionBoxPreserved, validQueueBox, and queueGrown are all true.
- L289 checks for the Activate transaction where:
    - L290 defines queueIn as the second input at index 1.
    - L291 defines collatBox as the second output at index 1.
    - L293 defines lenderPk as the SigmaProp in the queueIn box at R5.
    - L294 defines newEntry as the blake2b256 hash of the lenderPk SigmaProp proposition bytes.
    - L295 defines queueLIT as the LIT token at index 1 in the output queueOut box or has a fallback to a default value.
    - L296 defines collatLIT as the LIT tokens distributed for the current block emission at index 1 in the output collatBox or has a fallback to a default value.
    - L298 defines validQueueIn checking that:
        - The blake2b256 hash of the input queue box proposition bytes matches the CONST_GATE_HASH compile-time constant value.
        - The token id of the token at index 0 in the queue box is the same token id as the queue tokens in the emission input box.
    - L313 defines notAlreadyActive checking that the lenderPk associated with the input queue box is not already in the lenderSet.
    - L318 defines bootstrapping checking that the lenderSet size is below CONST_MAX_ACTIVE.
        - The process of the Activate spending path is for converting queue boxes to collateral boxes. This is the process of filling-up the lender set for the particular block.
    - L320-L339 defines setUpdated checking that:
        - If the bootstrapping condition is true, then the lender set in the output emission box contains the new lender pk hash.
        - Otherwise, the lender set is full, but if there is a retiring lender then he can be replaced with the current queue box:
            - L324 defines posBox as the proof-of-spend box being the third input at index 2.

- L325 defines retiring as the hash of the retiring lender pk in R4 of the proof-of-spend input box.
- All of the following must be true:
    - The blake2b256 hash of the posBox proposition bytes must equal the CONST_GATE_HASH compile-time constant.
    - The posBox token at index 0 must have as token id the same token id as the collateral tokens in the input emission box.
    - The posBox creation height must be less than the height of the block currently being validated.
    - The retiring lender is the lender in the lender set at index CTX_SLOT_IDX.
    - The updated lender set in the output emission box is the previous lender updated so that the retiring lender is replaced with the new lender corresponding to the input queue box.

- L345-L369 defines validCollatBox checking that:
    - The blake2b256 hash of the collatBox proposition bytes equals the CONST_COLLATERAL_HASH compile-time constant.
    - The first token at index 0 of the collatBox has the same token id as the collatToken id of the input emission box and there is only 1 in the collatBox output.
    - The output collatBox has at most 2 different tokens.
    - The LIT tokens in the collatBox have a token id equal to the CONST_LIT_ID compile-time constant or the sum of the emittedTotal and LIT tokens in the queue input box is 0.
        - Perhaps this was meant to check only the emittedTotal value, since the LIT permits provided by lenders in the queue box would need to be greater than 0, even after LIT emission is over?
    - The amount of LIT tokens in the collatBox is equal to the sum of the emittedTotal and LIT tokens in the queue input box.
    - R6 of the output collatBox contains a tuple of the public emission allocation and the founders set.
    - The value of the output collatBox is greater than or equal to the input queue box value.
    - R4, R5, and R7 of the output collatBox equals the same values as the input queue box.
    - R8 of the output collatBox contains the rollupHoldingHash bytes and the size is 32 bytes.
- L374-L382 defines litEmitted checking that:
    - If what is expected to be emitted is less than what is in the input emission box then:

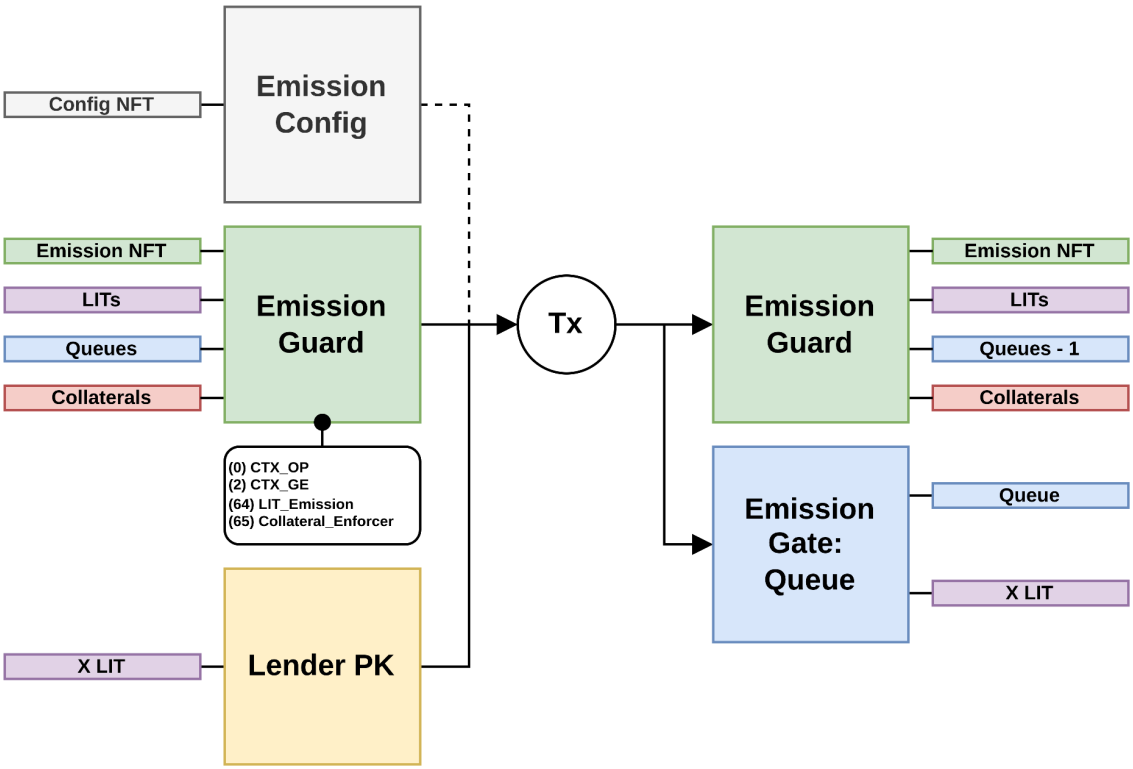
                - The output emission box has LIT tokens equal to the input amount minus the emitted amount.
                - The output emission box has a total of 4 unique tokens.
            - Otherwise, the output emission box has a total of 3 unique tokens.
        - L386-L398 defines queueAdvanced checking that:
            - The block counter in the output emission box is incremented by 1.
            - The total amount of queue tokens in the inputs should be equal to the amount of queue tokens in the input emission box plus the 1 queue token inside the input queue box.
            - If the lender set is in the bootstrapping phase, then:
                - The total amount of collateral tokens in the inputs should be equal to the number of collateral tokens in the input emission box and the output emission box should have one less collateral token.
            - Otherwise, if the lender set is full, then:
                - The total amount of collateral tokens in the inputs should be equal to the number of collateral tokens in the input emission box plus 1 collateral token from the proof-of-spend box and the output emission box should have the same amount of collateral tokens as the input emission box.
                    - The one from the input proof-of-spend box is reabsorbed but one is also given to the new output collateral box.
        - L400-L403 checks that emissionBoxPreserved, validQueueIn, notAlreadyActive, setUpdated, validCollatBox, litEmitted, and queueAdvanced are all true.
- L410 checks for the Clear transaction where:
    - L411 defines queueIn as the input queue box at index 1.
    - L412 defines lenderPk as the SigmaProp in R5 of the input queue box.
    - L413 defines entry as the blake2b256 hash of the lenderPk SigmaProp proposition bytes.
    - L415-L420 defines validQueueIn checking that:
        - The blake2b256 hash of the input queue box proposition bytes equals the CONST_GATE_HASH compile-time constant.
        - The input queue box has a token at index 0 whose token id equals the token id of the queue tokens inside the input emission box.
    - L423 defines lenderAlreadyActive checking that the lender set inside the input emission box contains the lender corresponding to the input queue box.
    - L425-L442 defines queueAdvanced checking that:
        - The block increment of the emission box does not change.
        - The lender set of the emission box does not change.
        - The total amount of queue tokens in the inputs equals the amount of queue tokens in the input emission box plus the 1 queue token from the input queue box.
        - The total amount of collateral tokens in the inputs equals the amount of collateral tokens in the emission box.

        - The amount of queue tokens in the output emission box is incremented by 1.
        - The amount of collateral tokens in the output emission box equals the amount of collateral tokens in the input emission box.
        - The LIT tokens of the output emission box equals the LIT tokens of the input emission box.
        - The output emission box has the same amount of unique tokens as the input emission box.
    - L444-L446 checks that emissionBoxPreserved, validQueueIn, lenderAlreadyActive, and queueAdvanced are all true.
- Otherwise, the contract evaluates to false.

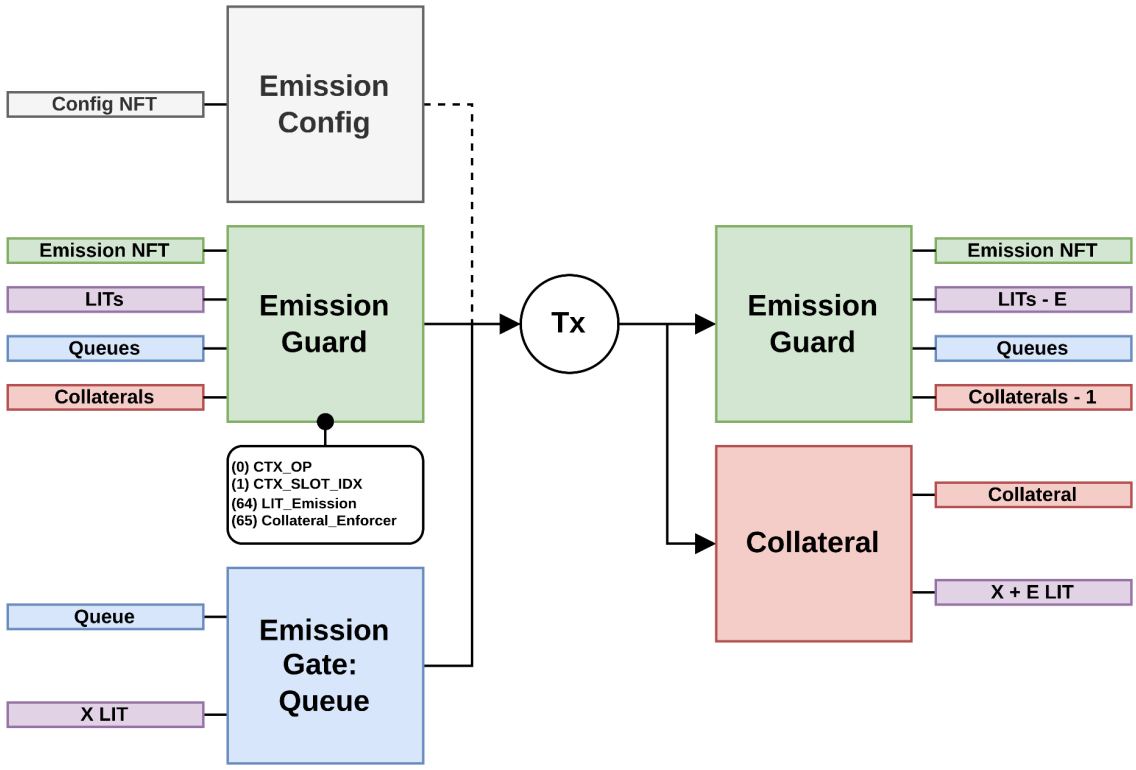
Transaction Diagrams

- The following diagrams apply for the Emission Config, Emission Guard, LIT Emissions, and Collateral Enforcer contracts.

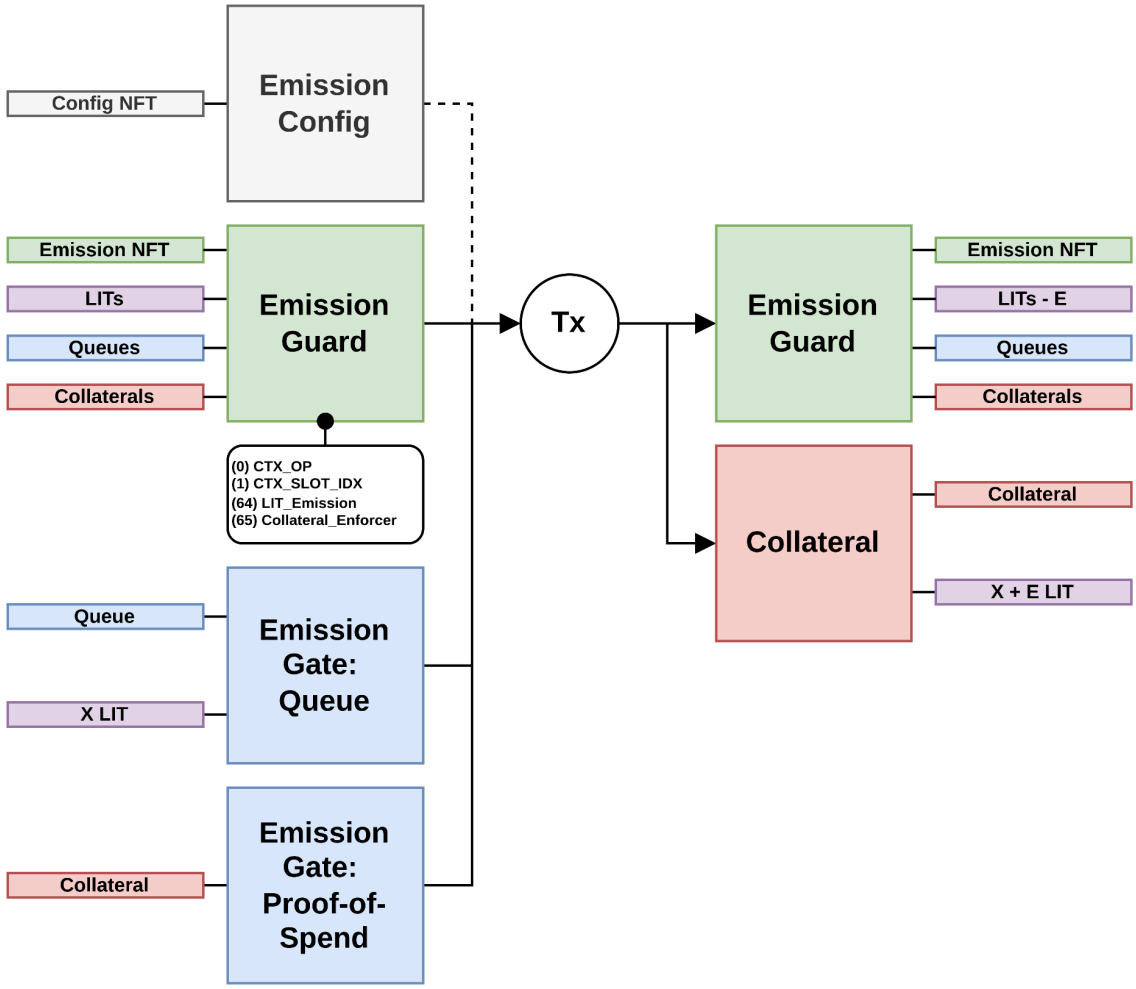
Emission: Join



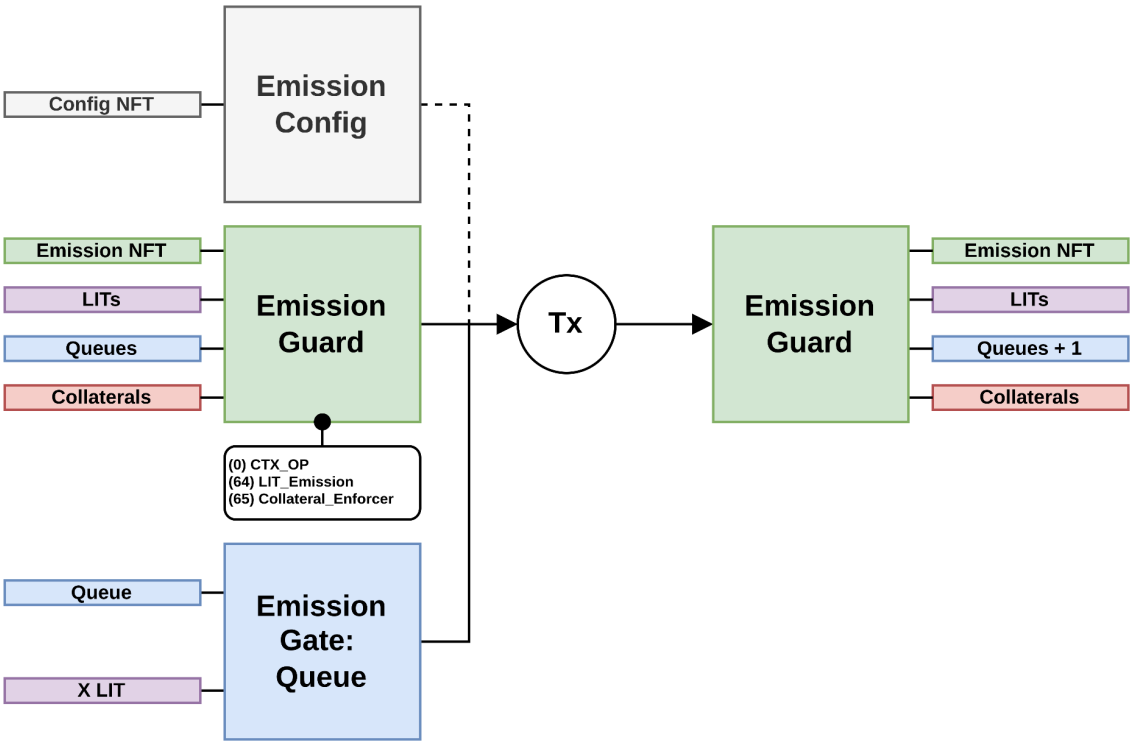
Emission: Activate - Bootstrap



Emission: Activate - No Bootstrap



Emission: Clear



Review Comments

- It is possible for the permit parameters referenced in the collateral enforcer to make the permit requirement be 0 (via the emission config registers).

Emission Config

Description

- Contains script hashes and parameters that can be updated and then referenced in the emissions contract.

Box Contents

Registers

- R4 contains a Coll[Long] representing LIT permit parameters, used for determining how many permits lenders are required to provide based on the queue backlog size:
    - Index 0: The floor.
    - Index 1: The ceiling.
    - Index 2: The slope.
- R5 contains a Coll[Byte] representing the hash of the collateral enforcer contract value bytes.
- R6 contains a Coll[Byte] representing the hash of the extension value bytes.
- R7 contains a Coll[Byte] representing the hash of the rollup contract proposition bytes.

Tokens

- Index 0: Contains the emission config NFT uniquely representing the emission config box.

Context Variables

- None

Compile-Time Constants

- CONST_TESTNET_PK: a SigmaProp representing the owner who can update the register values in the emission config box. In the mainnet deployment, this will be changed to a multi-sig.

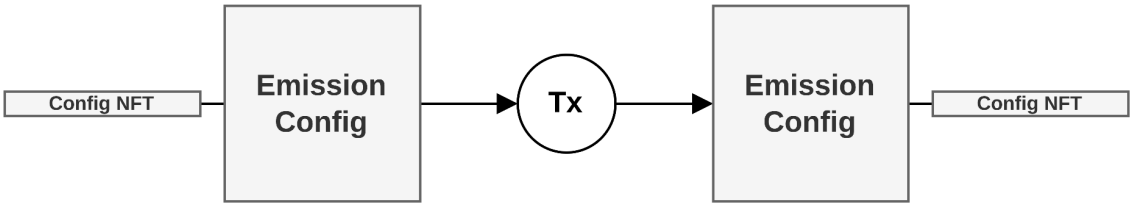
Contract Analysis

- L19-L24 defines variables representing the output emission config box at index 0
- L30-L41 defines recreated checking that:
    - The first input box at index 0 has the same id as the current emission config box. This ensures that there is one unique emission config box per transaction.
    - All registers values in the output emission config box are equal to the registers values in the input emission config box.
    - The output emission config box keeps the emission config NFT.

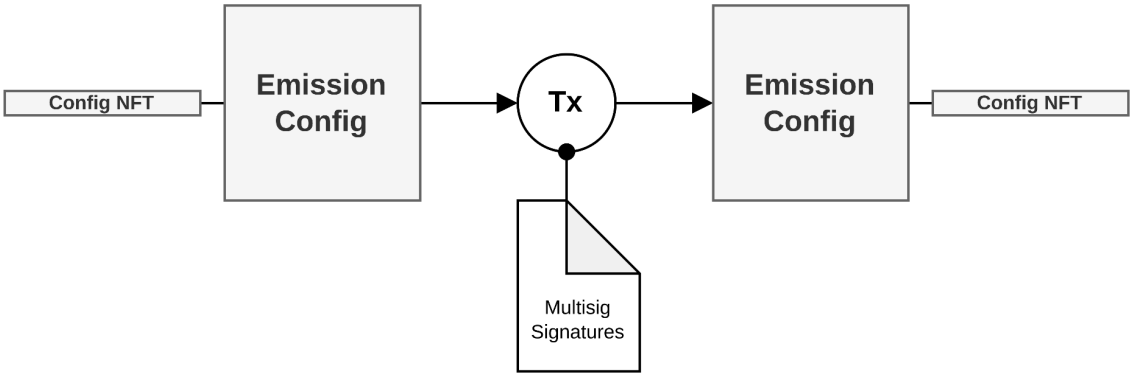
        - Redundantly checks the token amount and the token tuple separately. Could just keep the token tuple check assuming it is initialized with one token only.
    - The output emission config box keeps the same proposition bytes as the input emission config box.
- L42-L51 defines successorWellFormed checking that:
    - The output emission config box keeps the input emission config box NFT.
    - The output emission config box keeps the same proposition bytes as the input emission config box.
    - R4 of the output emission config box must have at least 3 LIT permit parameters.
    - R5, R6, and R7 must each have a total byte size equal to 32.
- L55 checks that recreated is true or that successorWellFormed and the CONST_TESTNET_PK are true.

Transaction Diagrams

Emission Config: Recreate



Emission Config: Update



Review Comments

- The multisig for the emission config box will include the project founders and some external contributors from the Ergo development community.

Collateral Enforcer

Description

- Contract checking that the collateral box gets created properly from a queue box.
- Executed from the transaction context along with the LIT emissions contract by the Emissions Guard box.

Box Contents

Registers

- None

Tokens

- None

Context Variables

- References CTX_OP at index 0 from emissions (guard) input box at index 0.
- References CTX_GE at index 2 from emissions (guard) input box at index 0.

Compile-Time Constants

- CONST_LIT_ID: a Coll[Byte] representing the LIT token id.

Contract Analysis

- L51 defines emissionConfig referencing the first data-input at index 0.
- L52 defines CONST_BLOCK_REWARD as a Long equal to 3_000_000_000 nanoErg.
- L53 defines CONST_FEE_CAP as a Long equal to 90_000_000 nanoErg.
- L54 defines CONST_DUST_BUDGET as a Long equal to 5_000_000 nanoErg.
- L57 defines nextEmission as the output emission box at index 0.
- L60 defines nextHead as the R6 Long value in the output emission box, representing the new queue head position.
- L61 defines nextTail as the R7 Long value in the output emission box, representing the new queue tail position.
- L67 checks for the Join transaction where:
    - L68 defines queueOut as the output queue box.
    - L73 defines queueGrown checking that:
        - The nextHead position equals the queueHead position from the input emission box. A queue is FIFO, so when an element gets added to the queue, the tail gets longer but the head remains the same.

        - The nextTail position equals the queueTail position from the input emission box incremented by 1.
    - L79-L82 accesses the permitParams from R4 of the emissionConfig data-input.
    - L83 defines backLog as the difference between queueTail and queueHead. This represents how many outstanding queue boxes there are that have not yet become collateral boxes.
    - L84 defines scaled as the value of the permitFloor variable plus the product of the permitSlope and backLog variables.
        - This variable defines how many LIT permit tokens a lender must provide in order to provide their ERG as collateral.
        - It follows a linear scale, such that a larger queue backlog leads to requiring more LIT permit tokens.
        - Thus, there is an incentive for lenders to join the queue, i.e. provide collateral, as soon as possible, since it will require them to provide fewer LIT permit tokens (lenders must purchase LIT tokens to provide as permits).
    - L85-L90 defines permitRequired checking that:
        - If scaled is greater than or equal to the permitCeil variable, representing the maximum amount of permits a lender is required to provide, then permitCiel is used.
        - Otherwise, scaled is used.
    - L92 defines queueLIT as the LIT tokens in the output queue box.
    - L93 defines permitPosted checking that the amount of LIT permit tokens in the output queue box equals the permitRequired amount.
    - L94 defines permitValid checking that the token id of the LIT permit tokens in the output queue box equals the CONST_LIT_ID compile-time constant.
    - L95 defines queueProp as the SigmaProp in R5 of the output queue box.
    - L96 defines queueBoxValid checking that:
        - R7 of the output queue box is a Long equal to the queueTail value of the input emission box.
            - Incrementing the tail tells us how many queue boxes have been created.
            - Incrementing the head tells us how many queue boxes have been consumed to create collateral boxes.
            - The difference between tail and head tells us the size of the active queue.
        - The proposition bytes of the output queue box equals the SigmaProp proposition bytes of the CTX_GE GroupElement.
        - The CTX_GE GroupElement is not the identity element.
        - The ERG value in the output queue box is at least the CONST_DUST_BUDGET plus the CONST_BLOCK_REWARD minus the CONST_FEE _CAP
            - The lender provides collateral at least equal to the Ergo blockchain block reward minus the fee they would get such that

when their collateral is used to pay miners, the Ergo blockchain reward will go to them and they will have their collateral + fee back.
        - R4 of the output queue box is a Long equal to the CONST_DUST_BUDGET value for creating extra boxes created in the protocol.
        - permitPosted is true.
        - permitValid is true.
        - R6 of the output queue box is a Byte equal to 0. It represents an extension flag.
    - L112 checks that queueGrown and queueBoxValid are true.
- L113 checks the Activate transaction where:
    - L115 defines queueIn as the second input box at index 1.
    - L116 defines queueAdvanced checking that:
        - R7 of the input queue box corresponds to the queueHead Long value in the input emission box. This makes sure that the queue box being processed is the correct one in the queue order, i.e. the head pointer has reached the current queue box.
        - nextHead in the output emission box is incremented by 1, indicating that the current queue box has been processed.
        - nextTail in the output emission box is equal to the queueTail value in the input emission box.
    - L134 checks that queueAdvanced is true.
- L124 checks the Clear transaction where:
    - L126 defines queueIn as the second input box at index 1.
    - L127 defines queueCleaned checking that:
        - R7 of the input queue box corresponds to the queueHead Long value in the input emission box. This makes sure that the queue box being processed is the correct one in the queue order, i.e. the head pointer has reached the current queue box.
            - So a clear/refund must still occur in the correct queue order.
        - nextHead in the output emission box is incremented by 1, indicating that the current queue box has been processed.
        - nextTail in the output emission box is equal to the queueTail value in the input emission box.
            - In the LIT Emissions contract, it is stated that the collateral enforcer contract could dictate where the lender's LIT permit tokens could go. Nothing here says where they go, so where do they go?

Review Comments

- For the Clear transaction, anyone can claim the LIT permit tokens and ERG inside the invalid queue box. This punishes people who try to spam the queue maliciously because it takes actual transaction fees and computational cost to clear the queue. The Lithos client makes sure to scan all queue, collateral, and proof-of-spend boxes before posting a new queue box.

Emission Gate

Description

- Contract used for both the Queue and Proof-of-Spend boxes. Register values differ depending on the box type.

Box Contents

Registers

- Queue:
    - R4: Fee value as Long.
    - R5: Lender PK as a SigmaProp.
    - R6: Extension flag as a Byte.
    - R7: Queue position as a Long.
- Proof-of-Spend:
    - R4: hashed proposition bytes of the lender as a Coll[Byte].

Tokens

- Queue:
    - Index 0: The queue token.
    - Index 1: The lender's LIT permit tokens.
- Proof-of-Spend:
    - Index 0: The collateral token proving the collateral box corresponding to the queue box was spent.

Context Variables

- None

Compile-Time Constants

- CONT_EMISSION_NFT: a Coll[Byte] representing the token id of the emission box NFT uniquely identifying it.

Contract Analysis

- L28 checks that the first input at index 0 contains the CONST_EMISSION_NFT.
    - So the first input is expected to be the emission box (emission guard contract). The LIT Emission contract executed from the transaction context will determine what type of box it expects this emission gate to be.

Review Comments

None

Emission Guard

Description

- The contract that is held inside the emission box that executes the emission contract and the collateral enforcer contract from the transaction context.

Box Contents

Registers

- Same as the LIT Emission contract.

Tokens

- Same as the LIT Emission contract.

Context Variables

- Index 64: CTX_EM_SCRIPT as a Coll[Byte] representing the serialized SigmaProp bytes of the emission contract.
- Index 65: CTX_ENFORCER_SCRIPT as a Coll[Byte] representing the serialized SigmaProp bytes of the collateral enforcer script.

Compile-Time Constants

- CONST_EM_SCRIPT_HASH: a Coll[Byte] representing the hashed value bytes of the emission script.
- CONST_EMCONFIG_NFT: a Coll[Byte] representing the token id of the emission config box NFT uniquely identifying it.
- CONST_EMISSION_NFT: a Coll[Byte] representing the token id of the emission box NFT uniquely identifying it.

Contract Analysis

- L17-L18 gets the context variables from index 64 and 65.
- L20 defines emissionConfig as the first data-input at index 0.
- L21 defines authenticConfig checking that CONST_EMCONFIG_NFT equals the token id of the first token at index 0 in the emissionConfig box, thus checking if there is a valid emission config box data-input provided.
- L22 defines enforcerScriptHash as a Coll[Byte] representing the hashed value bytes of the enforcer script, obtained from R5 of the emission config box. The emission config holds the enforcer script hash so that collateral box rules can be changed after the emission box is launched.
- L24 defines authenticEnforcerScript checking that the hash of the enforcer script context variable equals the hash provided by the emission config box data-input.

- L25 defines authenticEMScript checking that the hash of the emission script context variable equals the hash provided by the emission config box data-input.
- L26 defines authenticGuard checking that the token inside the current emission/guard box is the CONST_EMISSION_NFT compile-time constant value.
    - Though it seems vacuous, it ensures a duplicate emission guard box cannot exist, assuming there is only one emission NFT minted, but this could be verified on the Ergo blockchain itself.
- L27 checks that authenticConfig, authenticEMScript, authenticEnforcerScript, and authenticGuard are true, and that the executed emission and collateral enforcer scripts from the transaction context evaluate to true.

Review Comments

None

Collateral Mainnet

Description

- The contract containing the lender's provided collateral used to pay the miner for finding a block using the Lithos protocol. The collateral box can only be spent by a miner and it removes the lender from the emission box's active lender set.

Box Contents

Registers

- R4: Fee value as a Long taken from the block reward to fund other boxes.
- R5: Lender PK as a SigmaProp representing the lender that will receive the block's coinbase.
- R6: The LIT rewards as a tuple of a Long value representing the LIT emission for the block and a collection of tuples representing the founder addresses and their respective share allocation of the LIT emissions amount. These amounts were calculated by the emissions contract when creating the collateral box.
- R7: The extension flag as a Byte representing which collateral extension script must also be satisfied. Initially, this is set to 0 since there is no extension script at mainnet launch.
- R8: The hashed proposition bytes of the rollup-holding script as a Coll[Byte]. This is defined in the emission config box so it can be changed in the future, however, the collateral box keeps the same hash that was used to create it.

Tokens

- Index 0: The collateral token.
- Index 1: The LIT permit tokens + LIT emission tokens.
-

Context Variables

- Index 0: CTX_EXT_SCRIPT as a Coll[Byte] representing the serialized value bytes of the collateral extension script, read if there is an extension flag set.

Compile-Time Constants

- CONST_GATE_HASH: a Coll[Byte] representing the hashed proposition bytes of the emission gate contract, which is used to represent either a queue box or a proof-of-spend box.
- CONST_EMCONFIG_NFT: a Coll[Byte] representing the token id of the emission config NFT uniquely identifying the box.
- CONST_LIT_ID: a Coll[Byte] representing the token id of the LIT token.

Contract Analysis

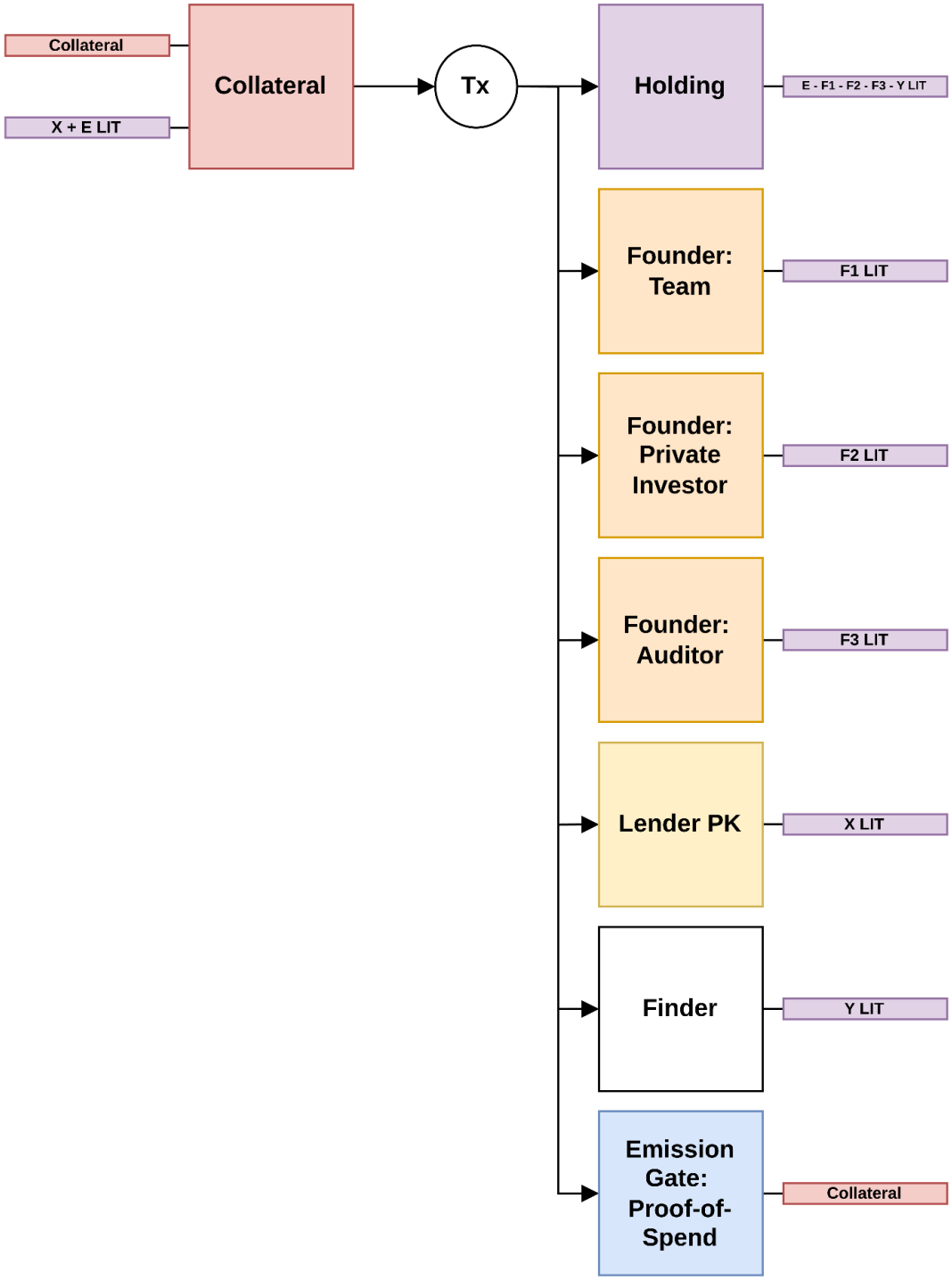
- L40 defines CONST_FINDER_DIVISOR as a Long equal to 25 used to give a fixed 4% of the LIT emission for the person who builds the genesis transaction.
- L44 defines holdingBox as the first output at index 0.
- L63-L67 gets the LIT block reward and private reward amounts from the set values in R6.
- L68 computers totalLITRewards as the sum of the lit block reward and the lit private reward.
- L71 defines permitChange as the difference between the amount of LIT tokens in the collateral box and the totalLITRewards calculated. This difference represents the lender's LIT permit tokens, which will be returned to the lender once the collateral box is spent.
- L73-L86 defines foundersPaid checking that:
    - If the LIT private rewards are greater than 0, then:
        - For each founder in the founder map stored in the collateral box register there exists a corresponding output such that:
            - There is a token at index 0 or fallback to a default value.
            - The blake2b256 hash of the box proposition bytes equals the founder proposition bytes.
            - The box value is greater than or equal to 1_000_000 nanoErg.
            - The token at index 0 has the CONST_LIT_ID compile-time constant as a token id and the amount is equal to the corresponding value from the founder map tuple element.
    - Otherwise, return true.
- L88-L99 defines permitChangeReturned checking that:
    - If permitChange is greater than 0, then:
        - There exists an output such that:
            - There is a token at index 0 or fallback to a default value.
            - The box proposition bytes equals the proposition bytes of the lender's SigmaProp stored in the register of the collateral box.
            - The box value is greater than or equal to 1_000_000 nanoErg.
                - Even though the min nanoErg per byte is a parameter that can hypothetically change in the future, if it is lowered for example, feeValue will increase and more will be taken from the block reward to create these boxes.
            - The token at index 0 has the CONST_LIT_ID compile-time constant as a token id and the amount is equal to permitChange.
    - Otherwise, return true.
- L112-L119 define proofOfSpendMade checking that there exists an output such that:
    - There is a token at index 0 or fallback to a default value.
    - The blake2b256 hash of the proposition bytes of the box equals the CONST_GATE_HASH compile-time constant, checking that a proof-of-spend box is created.

    - The token id equals the collateral token id in the collateral box and the amount is 1.
    - The creation-height of the proof-of-spend box equals the current height of the block being created.
    - R4 of the output proof-of-spend box is a Coll[Byte] equal to the blake2b256 hash of the lender's SigmaProp proposition bytes.
- L121-L140 defines extensionActivated checking that:
    - If the extensionFlag in the collateral box is not equal to 0, then:
        - Get the extension script CTX_EXT_SCRIPT from the transaction context at index 0, as a Coll[Byte].
        - The emission config is expected as a data-input at index 0.
        - L131 defines authenticConfig checking that the emission config box contains the CONST_EMCONFIG_NFT at index 0.
        - The extension script hash is obtained from the emission config at register R6.
        - L134 defines authenticExtension checking that the blake2b256 of the CTX_EXT_SCRIPT context variables equals the extension script hash from the emission config register R6.
        - L136 checks that authenticConfig and authenticExtension are true and that the CTX_EXT_SCRIPT executed from the transaction context evaluates to true.
    - Otherwise, return sigamProp(true).
- L148 defines finderLIT as the quotient of the LIT block rewards and the CONST_FINDER_DIVISOR, ensuring that whoever creates the genesis transaction gets 4% of the LIT block reward.
- L149 defines poolLIT as the LIT block reward minus the finderLIT, this is the LIT given to miners of the mining pool.
- L151-L158 defines tokensTransferred checking that:
    - If the poolLIT is greater than 0, then:
        - The holding box contains the poolLIT tokens at index 0.
    - Otherwise, return true.
- L160-L170 defines validAvlTree checking that R4 of the holding box contains the AvlTree where:
    - The digest is the empty digest.
    - All operations (insert, update, and remove) are allowed.
    - The key length is 32.
    - Values can be of different lengths.
- L172-L185 defines holdingValid checking that:
    - The holding box value is at least the value of the collateral box minus the feeValue used to create any other box in the outputs.
    - The blake2b256 hash of the proposition bytes equals the rollupHoldingHash.
    - tokensTransferred is true.
    - validAvlTree is true.
    - R5 of the holding box is an Int equal to 0.

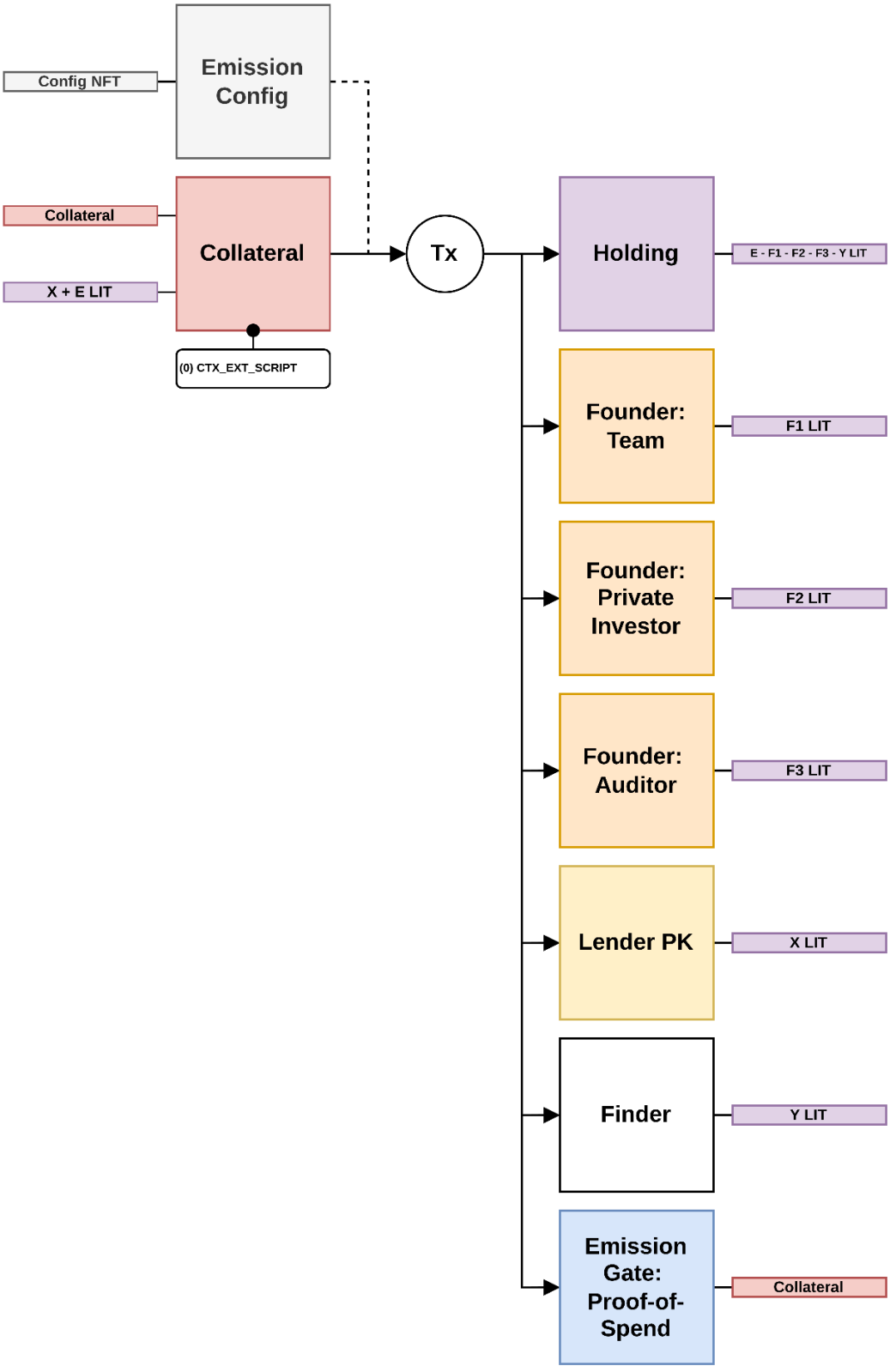
    - R6 of the holding box is a BigInt equal to 0.
    - R7 of the holding box is a Long equal to the HEIGHT of the current block being created that will include this genesis transaction.
    - R8 of the holding box is a Coll[Byte] with size equal to 32 representing the hash of the proposition bytes of the miner.
- L190 defines lenderIsBlockMiner checking that the lenderPk SigmaProp proposition bytes equals the SigmaProp proposition bytes of the minerPk from the block preHeader.
    - This ensures that the lender is the miner at the protocol level, ensuring they receive the full ERG block reward.
- L191 defines onlyOne checking that the first input at index 0 is the current collateral box, ensuring that no other collateral box exists in the transaction.
- L193-L202 defines validLithosBlock checking that:
    - holdingValid, permitChangeReturned, foundersPaid, proofOfSpendMade, lenderIsBlockMiner, and onlyOne are true.
- L204 checks that validLithosBlock is true and that extensionsActivated is true.

Transaction Diagrams

Genesis: No Extension



Genesis: With Extension



Review Comments

None

Appendix

Issue Guide

Severity	Description
CRITICAL	The contract contains one or more serious errors and warnings.
SERIOUS	An error that will cause unexpected behavior.
WARNING	A property that is questionable but will not cause unexpected behavior.
GOOD	Everything works as expected.

Status	Description
RESOLVED	The issues have been addressed. Default status if severity is GOOD.
UNRESOLVED	The issues have not been addressed.

About Us

4EYES Labs is an ErgoScript design and auditing firm for Ergo blockchain.

Linktree: https://linktr.ee/4EYESLabs